

Article

Three-Dimensional Path Planning of UAV Based on Improved Particle Swarm Optimization

Lixia Deng *, Huanyu Chen, Xiaoyiqun Zhang and Haiying Liu

School of Information and Automation Engineering, Qilu University of Technology (Shandong Academy of Sciences), Jinan 250353, China

* Correspondence: lixiadeng@qlu.edu.cn

Abstract: The traditional particle swarm optimization algorithm is fast and efficient, but it is easy to fall into a local optimum. An improved PSO algorithm is proposed and applied in 3D path planning of UAV to solve the problem. Improvement methods are described as follows: combining PSO algorithm with genetic algorithm (GA), setting dynamic inertia weight, adding sigmoid function to improve the crossover and mutation probability of genetic algorithm, and changing the selection method. The simulation results show that the improved PSO algorithm solves better route results and is faster and more stable.

Keywords: particle swarm algorithm; UAV; 3D path planning; SHADE algorithm

MSC: 93C85



Citation: Deng, L.; Chen, H.; Zhang, X.; Liu, H. Three-Dimensional Path Planning of UAV Based on Improved Particle Swarm Optimization. *Mathematics* **2023**, *11*, 1987. <https://doi.org/10.3390/math11091987>

Academic Editors: Haizhao Liang, Jianying Wang and Chuang Liu

Received: 17 March 2023

Revised: 8 April 2023

Accepted: 20 April 2023

Published: 23 April 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Currently, as robots enter our lives, boring, repetitive work is being transformed to a more unmanned and intelligent system using machines instead of work. Among them, the development of technologies related to drones has brought great convenience to our lives, such as inspecting and exploring dangerous environments, delivering deliveries, power patrols, and other tasks. For performing complex tasks in low-altitude flight, UAV navigation capabilities and planning paths are particularly important [1].

In 1959, Dr. Dantzig and Dr. Ramser first raised the vehicle-based routing problem. From then on, the routing problem has become a new research topic for domestic and foreign scholars [2]. Path planning algorithms suitable for UAV use can be divided into two categories, global path planning algorithms in the continuous domain range and local path planning algorithms in the continuous domain range [3]. In a two-dimensional environment, traditional algorithms such as A* algorithm [4], Dijkstra algorithm [5], and simulated annealing algorithm [6] perform search to find the optimal path by means of a raster map. However, when extended to three dimensions, the consumption of time and memory increases proportionally and is no longer applicable with path planning.

Sampling-based stochastic methods [7] have received wide application and attention in recent years. Random sampling in state space can effectively solve high-dimensional and complex path planning problems, but there are problems of low accuracy and slow convergence [8].

Inspired by nature, swarm intelligence algorithms are applied to route planning in complex environments, for example, particle swarm algorithm [9], ant colony algorithm [10], and genetic algorithm [11]. These algorithms find an efficient path through different strategies and then obtain the final path by continuous iterations. However, the tendency to fall into local optimum is still one of the problems that swarm intelligence algorithms need to solve.

Three-dimensional path planning is global path planning, which finds the best and collision-free path in 3D clutter considering geometric, physical, and temporal constraints [12].

This requires high accuracy and speed of the algorithm. The algorithms presented above have been applied to 3D path planning, among which the swarm intelligence algorithm stands out due to its excellent search capability. However, a single swarm intelligence algorithm still has limitations, and how to improve it has become one of the topics of scholars. Wang Yihu et al. [13] introduced the convergence and migration operations of the bacterial foraging algorithm (BFO) in the PSO algorithm, which effectively improved some of the defects of the PSO algorithm and improved its search capability. Xueying Sun et al. [14] proposed a high-performance bacterial foraging-genetic-particle swarm hybrid algorithm to address the defects of the particle swarm algorithm, which improved the computational speed and capability of the algorithm and further improved the usability of the method. B. Abhishek et al. [15] proposed a harmony-based search algorithm, which performs both exploratory search and usage search, and further optimizes the generated path combined with unmanned aerial vehicle constraints, while speeding up the algorithm to avoid falling into local optimum. Manh Duong Phung [16] presents a spherical quantum-oriented particle swarm optimization algorithm (SPSO), which transforms the path planning problem into an optimization problem containing the requirements and constraints for UAV's feasible and safe operation. The SPSO algorithm is used to find the optimal path through the relationship between particle position and UAV speed, turn angle, and pitch angle.

In this paper, the hybrid particle swarm optimization algorithm CPSO is designed to solve the UAV 3D path planning problem by combining the excellent design.

- Establish the experimental environment model, set 3D mountains as obstacles, construct fitness function based on obstacles and path length, and introduce cubic B-spline curve to smooth the path.
- Set the adaptive dynamic inertia weight of the adaptive particle swarm optimization algorithm to ensure the early search ability while enhancing the optimization ability of the later population.
- By introducing the selection operation of improved SHADE algorithm and improved genetic algorithm, the population diversity is improved, avoiding falling into local optima and reducing search time.
- Finally, comparative simulation experiments were conducted using MATLAB to compare with various swarm intelligence algorithms, including particle swarm optimization (PSO), particle swarm optimization algorithm with adaptive inertia weights (wPSO), SHADE algorithm (SHADE), genetic algorithm (GA), and ant colony algorithm (ACA), taking into account the overall performance of the algorithm.

The other parts of this article are arranged as follows: Section 2 establishes the experimental environment and path smoothing algorithm; Section 3 introduces the improvement strategy of CPSO in detail; Section 4 introduces the experimental environment, experimental results analysis, and algorithm comparison. Finally, the conclusion is given in Section 5, and the focus and improvement direction of follow-up work are proposed.

2. Model Establishment

This section describes the environmental model and path smoothing model used in this experiment, specifically for building a three-dimensional environmental model with mountain barriers and a cubic B-spline curve for smooth paths.

2.1. Environmental Model

The 3D path planning of the UAV needs to obtain information from the terrain model, and the actual situation should be considered when modeling the terrain. By considering obstacles, environment, and other factors, the established terrain model [17] is described as follows:

$$Z_1(x, y) = \sin(y + a) + b \cdot \sin(x) + c \cdot \cos(d \cdot \sqrt{x^2 + y^2}) + e \cdot \cos(y) + f \cdot \sin(g \cdot \sqrt{x^2 + y^2}) \quad (1)$$

where x and y are the horizontal coordinates, and Z_i are the corresponding height values. a, b, c, d, e, f , and g are constant coefficients that control the undulation of the base terrain and can be set as needed or generated randomly. For a mountain in 3D environment, it can be represented by the following model:

$$z(x, y) = \sum_{i=1}^P h_i \exp \left[- \left(\frac{x - x_i}{x_{si}} \right)^2 - \left(\frac{y - y_i}{y_{si}} \right)^2 \right] \quad (2)$$

where P represents the total number of mountain peaks, (x_i, y_i) represents the center coordinate of the i -th peak, and h_i is the parameter that controls the height. x_{si} and y_{si} are the attenuations of the i -th peak along the x -axis and y -axis which can be used to control the slope, respectively.

2.2. Path Smoothing Algorithm Based on Cubic B-Spline Curve

In order to prevent frequent angle adjustment during the flight, ensure the safety of the UAV, and reduce the sailing time, a cubic B-spline curve is introduced [18]. In a given $m + n + 1$ plane or space vertex $P_i (i = 0, 1, \dots, m + n)$, it is called a parametric curve segment of degree n :

$$P_{k,n}(t) = \sum_{i=0}^n P_{i+k} G_{i,n}(t) t \in [0, 1] \quad (3)$$

where $P_{k,n}(t)$ is the n -th degree B curve segment of the k -th segment, and these curve segments are called n -th degree B-spline curves. $G_{i,n}(t)$ is the basis function which is defined based on Equation (4).

$$G_{i,n}(t) = \frac{1}{n!} \sum_{j=0}^{n-i} (-1)^j C_{n+1}^j (t + n - i - j)^n \quad (4)$$

$$t \in [0, 1] i = 0, 1, \dots, n$$

in order to ensure the smoothness of the path and consider the difficulty, let $n = 3$, and a cubic B-spline curve is used to smooth the path.

3. Improved Particle Swarm CPSO Algorithm Design

This section describes the design ideas of the improved particle swarm optimization algorithm CPSO, including constructing a fitness function based on barriers and path length, changing fixed weights to adaptive dynamic weights to improve the optimization ability, and finally fusing the SHADE algorithm to improve population diversity.

3.1. Particle Swarm Algorithm

Particle swarm optimization (PSO) is an evolutionary computing technique. Inspired by the results of artificial life studies, Dr. Eberhart and Dr. Kennedy proposed a particle swarm algorithm by simulating bird foraging migration and population behavior and improving Craig Reynolds' bird cluster model [19]. A massless particle is designed to simulate a bird in a flock, and the particle has only two attributes: speed and position, where speed represents the speed of movement and location represents the direction of movement. Each particle individually searches for the optimal solution, records it as the current individual extreme value, and shares the individual extreme value with other particles in the whole particle swarm to find the optimal individual extreme value as the current global optimal solution for the whole particle swarm. All particles in the particle swarm adjust their speed and position according to the current individual extreme value they find and the current global optimal solution shared by the whole particle swarm. The basic idea of the particle swarm optimization algorithm is to find the best solution through

collaboration and information sharing among individuals in a group. The particle swarm optimization algorithm operates on particles using the following formulas:

$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1(t)[pbest_i - x_{ij}(t)] + c_2r_2(t)[gbest_i - x_{ij}(t)] \quad (5)$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1) \quad (6)$$

where w denotes the inertial weight and the degree of trust in the current speed direction. c_1, c_2 is the learning factor, also known as the acceleration constant; r_1, r_2 is a random value between 0 and 1, increasing search randomness. v_{ij} is the velocity of the particle. $pbest$ is the best position for the i particle to experience, and $gbest$ is the best position for all particles of the group to experience. x_{ij} is the current position of the particle.

3.2. Fitness Function Design

The quality of the path length is one of the important indicators to measure the success of the algorithm improvement. Due to the lack of battery capacity of the UAV, the flight distance is limited. The shorter the flight path, the less time and energy it takes.

Based on the cubic B-spline curve fitting path, the interpolation process is performed, and the interpolation is differentiated to obtain the fitness function:

$$fitness = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2 + (z_{i+1} - z_i)^2} \quad (7)$$

where (x_i, y_i, z_i) are the coordinates of the i node of the path; $(x_{i+1}, y_{i+1}, z_{i+1})$ are the coordinates of the $i+1$ node. The obstacle risk factor f is introduced to avoid the collision between the UAV and the obstacle. The barrier coefficient formula is described as follows:

$$f = \begin{cases} 0 & L_{\min} > L_d \\ 1 & L_{\min} < L_d \end{cases} \quad (8)$$

considering the real environment, UAV is not a particle and it has its own size. So, $L_{\min}$ is set as the minimum distance close to the peak, and L_d as the safe distance. Combined with laboratory drone data, set $L_d = 0.12$. When $f = 1$, the minimum distance is less than the safe distance, which is prone to danger, so it is necessary to increase the fitness function. The fitness function is changed to:

$$fitness = kfitness \quad (9)$$

where k is the multiple of expansion, which can be set according to the experimental environment. This experiment has set $k = 5$.

3.3. Adaptive Dynamic Inertial Weight

The inertia weight is an important control parameter in the particle swarm algorithm, and the size of the inertia weight indicates how much of the current velocity inheritance goes to the particle. If inertia weight is set larger, the global search ability is stronger, and if inertia weight is set smaller, the local search ability is stronger and the global search ability becomes weaker [20]. In this paper, a linearly decreasing inertia weight is designed. In the early stage of the algorithm, a larger inertia weight is used to ensure the global search ability. With the increase in number of iterations, the inertia weight becomes smaller and the local search ability is enhanced. The formula is described as follows:

$$w = (w_{\max} - w_{\min}) \times \frac{(N - iter)}{N} + \frac{(w_{\max} - w_{\min})}{iter} \quad (10)$$

where $w_{\max}$ is the maximum inertia weight, $w_{\min}$ is the minimum inertia weight, N is the maximum number of iterations, and $iter$ is the current iteration number of the algorithm.

3.4. Improved SHADE Algorithm

Differential evolution algorithm (DE) [21] belongs to one of the evolutionary algorithms (EA). It includes the following steps: 1. initialization of the population. 2. mutation operation. 3. crossover operation. 4. selection operation. Then after that, the JADE [22] algorithm is updated, which has the same logic as DE. Subsequently, SHADE [23] was introduced on the basis of JADE. In this paper, the DE family algorithm is chosen to increase the population diversity, prevent falling into local optimum, and at the same time improve the operation speed.

According to the characteristics of DE, JADE, and SHADE algorithms, the variation operation and crossover operation are taken to be used in the algorithm. The number of populations in this experiment is $M = 50$; if each iteration does not consider the interference of invalid points, this will lead to an increase in the algorithm running speed. By the selection operation of hybrid genetic algorithm GA, the good individuals are left and the bad ones are eliminated.

This paper adopts the mixed selection operator. The first method uses the optimal fitness selection method to sort the fitness and selects the better fitness as the parent 1, with selection of populations at 50%. The second method uses the roulette method by selecting the probability p_{sec} . The selected population is used as parent 2, and the population share is 50%. The combination becomes the parent of the next iteration.

The variation operation achieves individual variation through a difference strategy, and the improved variation strategy is chosen in this paper: DE/current-to-best/1. The equation is as follows:

$$V_{i,j,v}(g+1) = X_{i,G} + F_i \cdot (X_{\text{best},G}^P - X_{i,G}) + F_i \cdot (X_{r_1,G}^i - X_{r_2,G}^i) \quad (11)$$

$$\mu_F = (1 - c) \cdot \mu_F + c \cdot \text{mean}(S_F) \quad (12)$$

$$F_i = \text{rand}(\mu_F, 0.1) \quad (13)$$

where $X_{i,G}$ is the i particle being processed, $X_{\text{best},G}^P$ is an individual in the top $p * M$ of the current population fitness ranking, p is a given proportion, $p = 10\%$, $X_{r_1,G}^i$ is the i particle being processed of parent 1 selected by GA, $X_{r_2,G}^i$ is the i -th particle being processed of parent 2 selected by GA, $\mu_F = 0.4$, F_i is the scale factor, $c = \frac{1}{10}$, and $\text{mean}(S_F)$ is the ratio of parent optimal fitness function to population size.

Afterwards, crossover operations between individuals were performed to improve population diversity for the populations after mutation operations. The formula is as follows:

$$u_{j,i,v}(g+1) = \begin{cases} v_{j,i,v}(g+1), & \text{if } \text{rand}(0,1) \leq C_r \\ x_{j,i,v}(g+1), & \text{otherwise} \end{cases} \quad (14)$$

$$\mu_{CR} = (1 - c) \cdot \mu_{CR} + c \cdot \text{mean}(S_{cr}) \quad (15)$$

$$C_r = \text{rand}(\mu_{CR}, 0.1) \quad (16)$$

where $x_{j,i,v}(g+1)$ is the parent population, C_r is the crossover probability, $\mu_{CR} = 0.5$, $c = \frac{1}{10}$, and $\text{mean}(S_{cr})$ is the ratio of the current population optimal fitness function to the population size.

3.5. Constraint Condition

In order to prevent the UAV from being dangerous during flight, constraints need to be set according to the actual situation. Firstly, the altitude has a great influence on the UAV. Flight at high altitude is susceptible to temperature and airflow, and flight at low altitudes is susceptible to disturbances from buildings and trees. Therefore, flying at the appropriate altitude can be expressed as Equation (13):

$$z_{\min} < z_j < z_{\max} \quad (17)$$

where z_j is the height position of the j -th time. $z_{\min}, z_{\max}$ represent the minimum and maximum heights, and $z_{\min} = 5, z_{\max} = 100$ is set according to the actual situation. Meanwhile, the size of environment is set to $100 \times 100 \times 250$ m to prevent the UAV from flying out of the set environment.

4. Experimental Simulation and Analysis

Firstly, summarize the algorithm flowchart according to the third section, and then introduce the experimental hardware configuration and some algorithm parameters of this experiment. Perform comparative analysis of algorithms in the same and random environments to validate the improved particle swarm optimization algorithm.

4.1. Improved Algorithm Operation Flow

The flow chart of the improved PSO algorithm is presented in Figure 1.

(1) Establishing an experimental environment according to Equations (1) and (2); refer to Section 2.1. Setting the start point and end point. The starting point is represented as a box and the ending point is an asterisk.

(2) Parameter initialization. Setting the particle population size, maximum number of iterations, inertia weight, social weight, and cognitive weight. For parameter settings, refer to Table 1.

(3) Population initialization. Randomly generating particles and initializing the velocity, calculating the initial fitness and performing collision detection, and updating the individual optimum as well as the global optimum.

(4) Enter the main loop. Update velocity and position, perform velocity and position detection at the same time to avoid out-of-bounds, calculate fitness values and perform collision detection, update individual optimum and global optimum.

(5) The selection operation of genetic algorithm is introduced to select the outstanding particle population as the parent, followed by the crossover and mutation operation of SHADE algorithm. The new population is used as the initial population for the next performed cycle.

(6) End condition. Determine if the maximum number of iterations has been reached, and if so, exit the loop and output the result; otherwise, return to step (4).

4.2. Experimental Environment and Parameters

To verify the advantages of the proposed CPSO algorithm, traditional PSO algorithm, SHADE, and PSO with modified dynamic inertia weights (wPSO) are used as control group, and the parameters such as the number of iterations remain unchanged. The above three algorithms are simulated and tested on MATLAB. Two sets of experiments are carried out and the experimental results are analyzed. The test environment is Windows 10, 64-bit system, MATLAB R2020b simulation platform. Parameters in the algorithm are shown in Table 1:

In order to verify the superiority of the improved PSO algorithm in 3D path planning, the following two experiments are carried out, respectively.

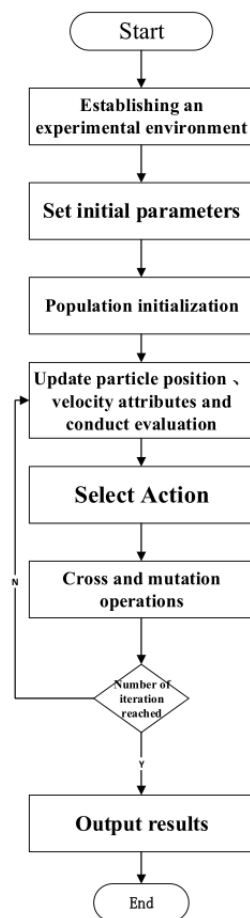


Figure 1. Flow chart.

Table 1. Algorithm parameters.

Quantity	Symbol	Numerical Value
Spatial scope	/	$100 \times 100 \times 250$
Starting point	start	(1,1,1)
End point	goal	(95,76,30)
Total group number	M	50
Number of iterations	N	200
Current iterations	$iter$	/
Social weight	c_1	2
Cognitive weight	c_2	2
Maximum inertia weight	$w_{\max}$	0.9
Minimum inertia weight	$w_{\max}$	0.4
Expand multiple	k	5

4.3. Comparative Analysis in the Same Simple Environment

This section conducted comparative analysis experiments on algorithms in simple and complex environments. Figure 2 shows the front view of 3D path planning results in a simple environment, and Figure 3 shows the front view of 3D path planning results in a complex environment. Figures 4 and 5 show the fitness curves of simple and complex environments, respectively.

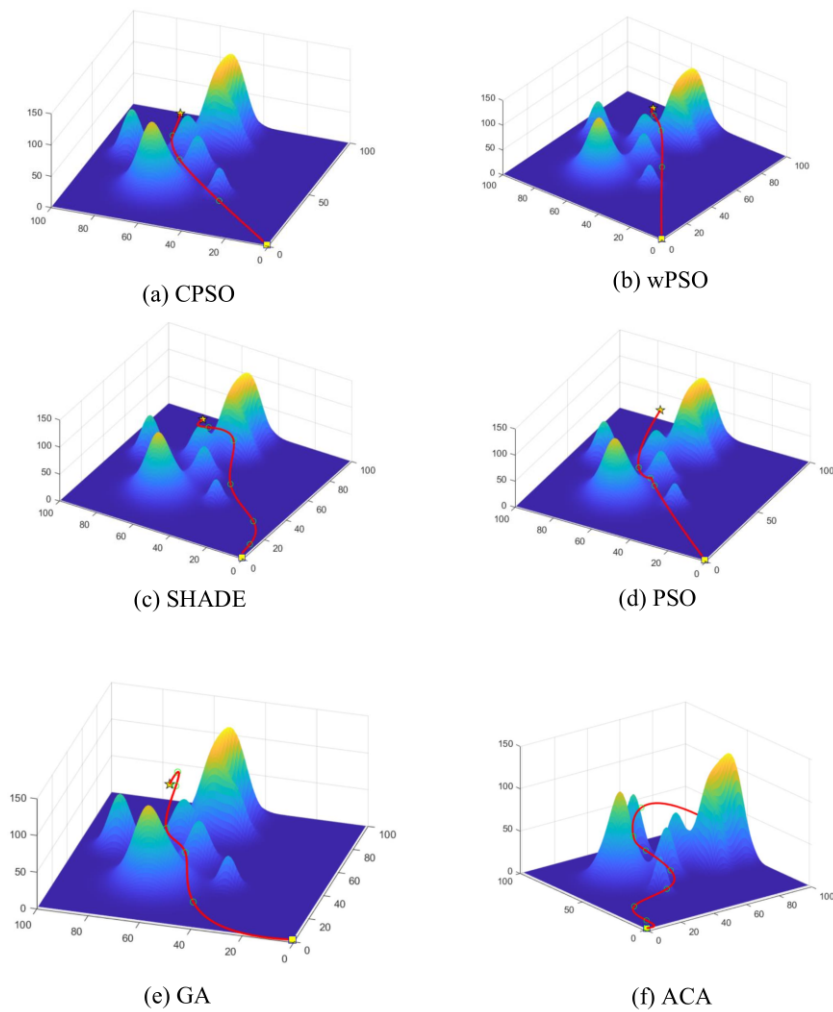


Figure 2. Simple environment result diagram.

From Figures 2 and 3, it can be seen that several algorithms can complete path planning tasks in a three-dimensional environment. From the e and f graphs in Figure 3, it can be observed that the GA and ACA algorithms generate long paths with multiple inflection points, and although they can stay away from obstacles, they are not the optimal choice. Compared to e and f graphs, the path generation of C graph is better, but there is still a significant curve situation. The generation path of D graph is excellent, but the inflection points can be clearly found in the graph. The simple PSO algorithm is not sufficient for path smoothness, so it is not adopted. The path generation in Figure 2b is smooth, but compared to Figure 2a, the degree of path optimization is clearly not excellent enough. Overall, the CPSO generated in Figure 2a has a smooth path, no inflection points, good continuity, and the shortest path.

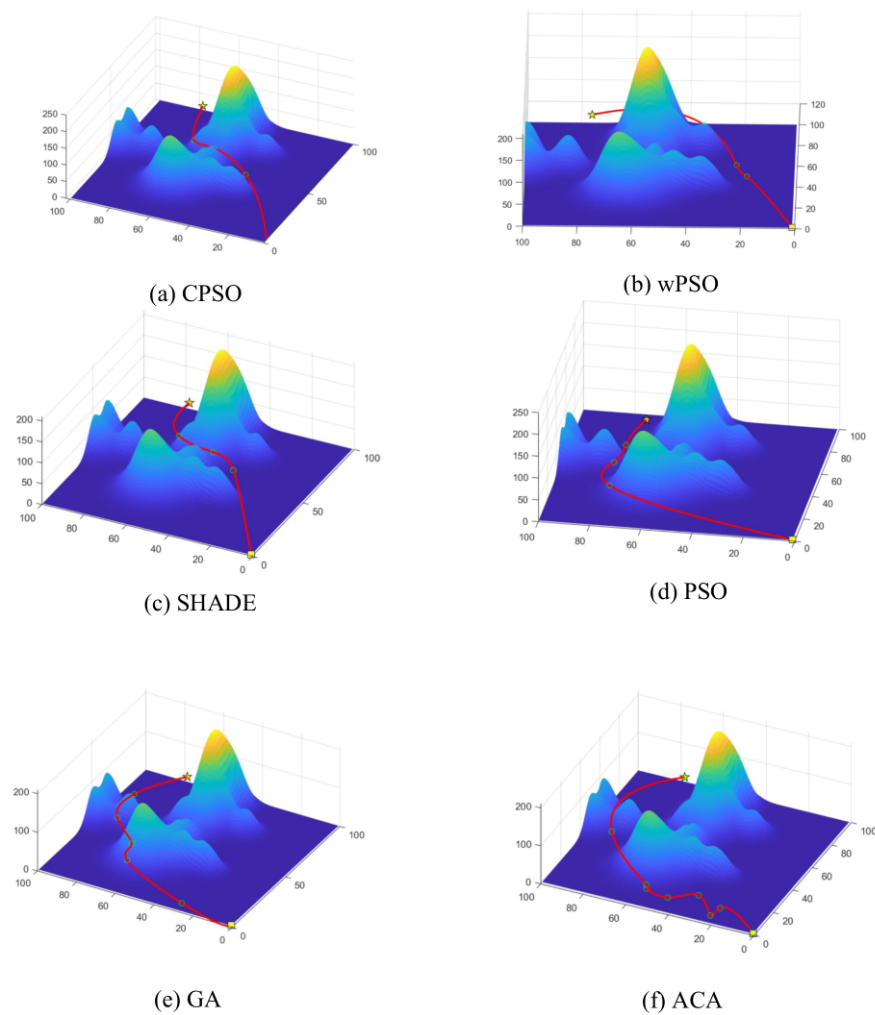


Figure 3. Complex environment result diagram.

From the fitness curve in Figure 6, it can be seen that the PSO algorithm and wPSO algorithm have been trapped in local optimum around 20 iterations, lacking the ability to jump out of local optimum, among which PSO algorithm has been trapped in local optimum in the ninth iteration. Although the search was still in progress at the 115th iteration, the effect was no longer apparent. Because of the inertial dynamic weights, wPSO reaches its optimum in about 20 generations and still tries to optimize in later generations, but the diversity of the population is insufficient, resulting in poor search ability in later generations. Because the SHADE algorithm needs to be continuously evolved, it takes a long time to reach its optimal level in about 60 generations, and the performance of the algorithm is not as good as other control groups. GA and ACA algorithms have large fitness and weak optimization ability, which require great computing power for 3D path optimization tasks, so there is still great room for the development of the two algorithms. The CPSO algorithm incorporates dynamic inertia weights, which are larger in the pre-iteration period; it guarantees global searching ability and sharply reduces the fitness curve. As the number of iterations increases, the weight decreases, the local optimization ability is strengthened, and the convergence rate is increased, and the optimal result is basically reached in 23 generations. The selection operation in the genetic algorithm and the crossover and mutation operation in SHADE are introduced to improve the diversity of the population, enhance the search ability, and still perform local optimization in the late iteration.

For the complex path planning in Figure 3, the differences in path generation are particularly prominent. Firstly, the drawbacks of Figure 3e,f in simple obstacles are magnified in complex obstacles, resulting in more inflection points and longer generation paths, proving that the two algorithms have poor processing ability in complex three-dimensional environments. The d and b graphs perform well in simple environments, but their own shortcomings are exposed when the environment is complex. Firstly, there is a problem of generating path differences in both Figure 3b,d, and there are large-scale curves, which can lead to the risk of drone crashes during flight. In addition, there are also a few inflection points in the paths generated by the two, further increasing the danger of drone flight. From the image, CPSO and SHADE are both excellent for complex 3D path planning problems, but from the fitness Figure 5, it can be found that CPSO's fitness curve is significantly lower than SHADE's, so the path generated by CPSO is better.

In summary, in increasingly complex environments, CPSO performs significantly better than other comparative algorithms, so CPSO can be used as a choice for dealing with path planning problems in complex environments.

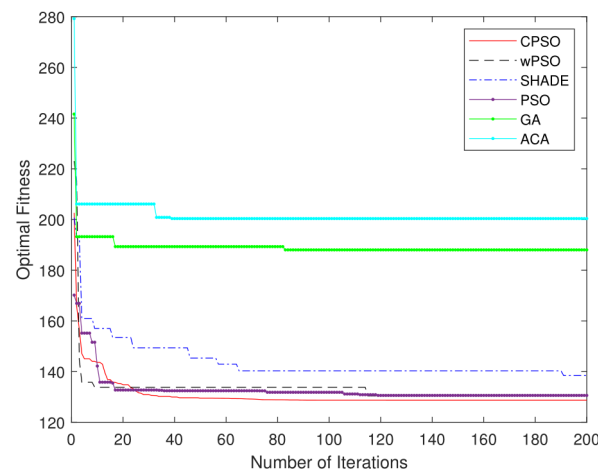


Figure 4. Simple environmental fitness value.

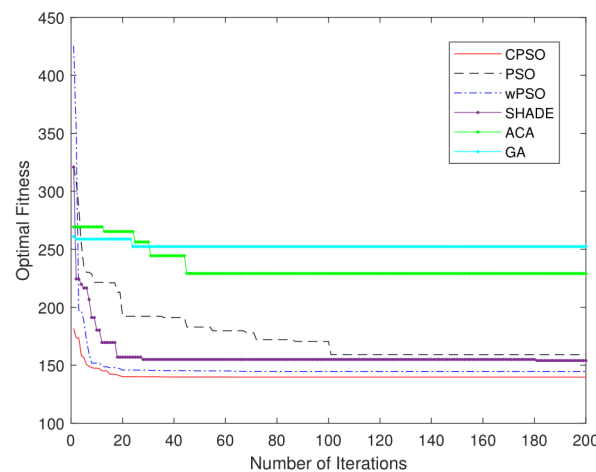


Figure 5. Complex environmental fitness value.

4.4. Comparative Analysis in Random Environment

Establish a three-dimensional random environment within the range of $100 \times 100 \times 100$ m, with a starting point of (1,1,1) m and an endpoint of (100,100,50) m. Randomly generate 10 obstacles with 200 iterations. Perform 10 simulation tests on the 6 algorithms mentioned above, and the simulation results are shown in Figure 6. The statistical results are

shown in Table 2. It can be found from the statistical table that the average fitness of CPSO algorithm is significantly lower than that of other comparison algorithms, showing good optimization ability. Through variance comparison, CPSO has good stability. According to Table 2, by comparing the number of iterations and time to reach the average fitness value, it can be found that CPSO can reach the average fitness in a relatively short time. The number of iterations is 14 generations, and it takes 14.48 s. The traditional PSO algorithm requires 37 iterations and takes 12.84 s. The SHADE algorithm and wPSO require 26 and 37 iterations, consuming 14.21 s and 7.21 s. GA and ACA are obviously poor in fitness and higher than other algorithms. In summary, it can be proven that the CPSO algorithm is more stable and faster than other algorithms.

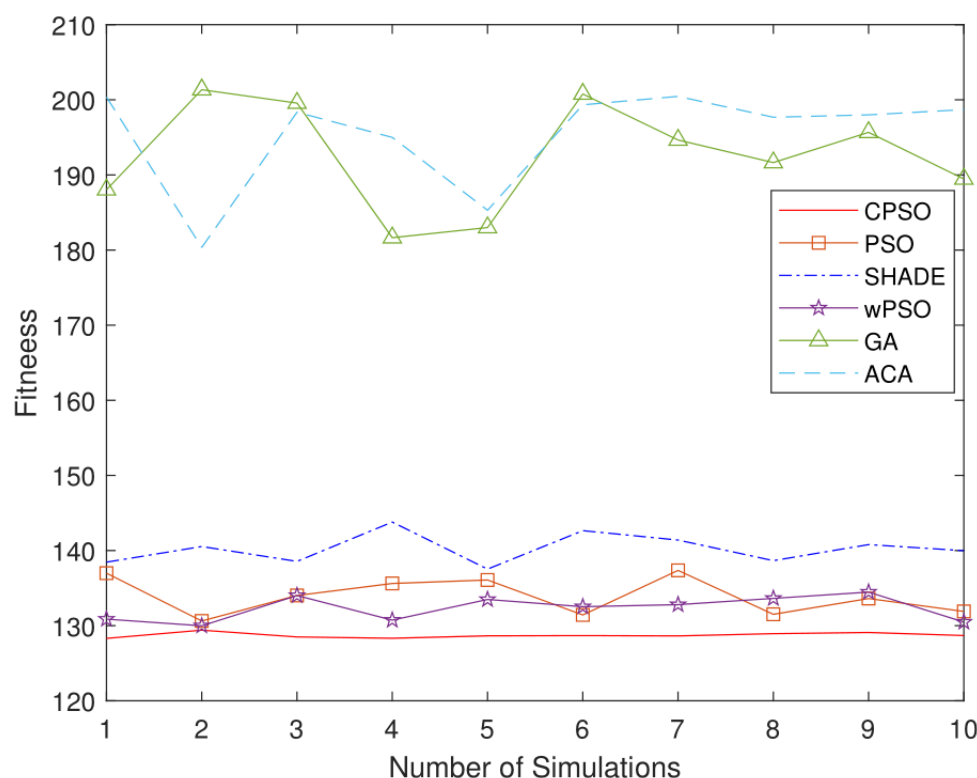


Figure 6. Ten simulation results.

Table 2. Simulation result statistics.

Algorithm	Average Fitness Value	Average Running Time/s	Fitness Value Variance	Average Number of Iterations to Reach Average Value	Time to Reach Average Fitness
CPSO	128.723	50.56	0.11	14	3.5
PSO	135.314	69.38	6.21	37	12.84
SHADE	140.237	90.45	4.011	60	27.14
wPSO	132.301	55.49	2.69	26	7.21
GA	192.585	85.28	44.941	34	14.5
ACA	195.355	275.38	47.337	39	53.7

5. Conclusions

Aiming at the shortcomings of the traditional PSO algorithm, with which it is easy to fall into local optimum, this paper proposes an improved PSO algorithm and applies it to 3D path planning. The improvement method is: introducing dynamic inertia weight,

adding selection operation in genetic algorithm, adding crossover and mutation operation in SHADE algorithm, using mixed selection operations, introducing crossover and mutation operations, increasing the diversity of the population, retaining the global search ability of the particle swarm optimization algorithm, and enhancing the local search ability in the later stage of iteration. The CPSO algorithm, PSO algorithm, GA algorithm, and other algorithms are simulated and tested by MATLAB. The test results show that the improved PSO algorithm has better searching ability and stability, and performs significantly better in a complex environment than the comparison algorithm. Compared with PSO, GA, and other algorithms, the CPSO algorithm generates a shorter path length and smoother path, which improves the quality and efficiency of UAV routing. The CPSO average fitness value increased by about 5.12% compared with PSO, 49.61% compared with GA, 2.78% compared with wPSO, 8.94% compared with SHADE, and 51.76% compared with ACA.

This experiment verifies the feasibility of the algorithm through simulation, and assumes that the external environment has no influence. Future research will focus on the optimization of UAV three-dimensional path planning algorithms in real and dynamic environments and make them available for daily use.

Author Contributions: Writing—original draft, H.C.; Validation, X.Z.; Resources, H.L.; Writing—review and editing, L.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by ZR2018QF005.

Conflicts of Interest: The authors declare no conflict of interest.

References

- Nonami, K. Present state and future prospect of autonomous control technology for industrial drones. *IEEE Trans. Electr. Electron. Eng.* **2020**, *15*, 6–11. [\[CrossRef\]](#)
- Giagkiozis, I.; Purshouse, R.C.; Fleming, P.J. An overview of population-based algorithms for multi-objective optimisation. *Int. J. Syst. Sci.* **2015**, *46*, 1572–1599. [\[CrossRef\]](#)
- Deng, L.; Chen, H.; Liu, H.; Zhang, H.; Zhao, Y. Overview of UAV path planning algorithms. In Proceedings of the 2021 IEEE International Conference on Electronic Technology, Communication and Information (ICETCI), Changchun, China, 27–29 August 2021; pp. 520–523. [\[CrossRef\]](#)
- Hart, P.E.; Nilsson, N.J.; Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Trans. Syst. Sci. Cybern.* **1968**, *2*, 100–107. [\[CrossRef\]](#)
- Dijkstra, E.W. A note on two problems in connexion with graphs. *Numer. Math.* **1951**, *1*, 269–271. [\[CrossRef\]](#)
- Steinbrunn, M.; Moerkotte, G.; Kemper, A. Heuristic and randomized optimization for the join ordering problem. *VLDB J.* **1997**, *6*, 191–208. [\[CrossRef\]](#)
- LaValle, S.M.; Kuffner, J.J. Randomized Kinodynamic Planning. *Int. J. Robot. Res.* **1999**, *15*, 378–400.
- Wang, J.; Meng, Q.H. Optimal Path Planning Using Generalized Voronoi Graph and Multiple Potential Functions. *IEEE Trans. Ind. Electron.* **2020**, *67*, 10621–10630. [\[CrossRef\]](#)
- Hua, X. Phase-out factor with particle swarm optimization. In Proceedings of the Second International Conference on Mechanic Automation and Control Engineering, Inner Mongolia, China, 17 July–15 July 2011.
- Dorigo, M.; Gambardella, L.M. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Trans. Evol. Comput.* **1997**, *1*, 53–66. [\[CrossRef\]](#)
- Holland, J.H. Outline for a Logical Theory of Adaptive Systems. *J. ACM* **1962**, *9*, 297–314. [\[CrossRef\]](#)
- Yang, L.; Qi, J.; Xiao, J.; Yong, X. A literature review of UAV 3D path planning. In Proceedings of the 11th World Congress on Intelligent Control and Automation, Shenyang, China, 29 June–4 July 2014.
- Wang, Y.H.; Wang, S.M. UAV Path Planning Based on Improved Particle Swarm Optimization Algorithm. *Comput. Eng. Sci.* **2020**, *42*, 7.
- Samuel, G.G.; Rajan, C.C.A. Hybrid Particle Swarm Optimization—Genetic algorithm and Particle Swarm Optimization—Evolutionary programming for long-term generation maintenance scheduling. In Proceedings of the 2013 International Conference on Renewable Energy and Sustainable Energy (ICRESE), Coimbatore, India, 5–6 December 2013.
- Abhishek, B.; Ranjit, S.; Shankar, T.; Eappen, G.; Rajesh, A. Hybrid PSO-HSA and PSO-GA algorithm for 3D path planning in autonomous UAVs. *SN Appl. Sci.* **2020**, *2*, 1–16. [\[CrossRef\]](#)
- Phung, M.; Ha, Q.P. Safety-enhanced UAV path planning with spherical vector-based particle swarm optimization. *Appl. Soft Comput.* **2021**, *107*, 107376. [\[CrossRef\]](#)
- Qi, Z.; Shao, Z.; Ping, Y.S.; Hiot, L.M.; Leong, Y.K. An Improved Heuristic Algorithm for UAV Path Planning in 3D Environment. In Proceedings of the International Conference on Intelligent Human-Machine Systems & Cybernetics, Nanjing, China, 26–28 August 2010.

18. Kennedy, J.; Eberhart, R. Particle Swarm Optimization. In Proceedings of the Icn95-International Conference on Neural Networks, Perth, WA, Australia, 27 November–1 December 1995.
19. Wang, X.; Meng, X. UAV Online Path Planning Based on Improved Genetic Algorithm. In Proceedings of the Chinese Control Conference (CCC), Guangzhou, China, 27–30 July 2019.
20. Wai, R.-J.; Prasetya, A.S. Adaptive Neural Network Control and Optimal Path Planning of UAV Surveillance System With Energy Consumption Prediction. *IEEE Access* **2019**, *7*, 126137–126153. [[CrossRef](#)]
21. Tanabe, R.; Fukunaga, A. Success-history based parameter adaptation for Differential Evolution. In Proceedings of the 2013 IEEE Congress on Evolutionary Computation (CEC), Cancun, Mexico, 20–23 June 2013.
22. Zhang, J.; Sanderson, A.C. JADE: Adaptive Differential Evolution with Optional External Archive. *IEEE Trans. Evol. Comput.* **2009**, *13*, 945–958. [[CrossRef](#)]
23. Tanabe, R.; Fukunaga, A.S. Improving the search performance of SHADE using linear population size reduction. In Proceedings of the Evolutionary Computation, Beijing, China, 6–11 July 2014.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.